SLE-4: Architecture Decision Record (ADR)

Course: 02AML204 — Introduction to Artificial Intelligence

PRN	25UAM085
Name	Sharvil S. Chavan
Division	B
Date	08 October 2026
GitHub / Portfolio Link	https://github.com/SharvilChavan-10

PART A: Architecture Decision Record (ADR)

Title	Use of Breadth-First Search (FIFO Queue + Visited Set + Parent Map) as the Primary Maze-Solving Strategy
Status	Accepted
Date	08 October 2026
PRN & Name	25UAM085 — Sharvil S. Chavan

1. Context

In SLE-2 and SLE-3 I built a Maze Solver System that generates grid mazes (20×20, 40×40, 70×70) with walls, one start cell and one goal cell, and must return a path from start to goal. I had to decide which search strategy should be the system's primary solver, because the two uninformed algorithms I implemented behave very differently: DFS is cheap but returns long, winding paths, while BFS costs more but returns shortest paths. This choice decides how the Frontier, Visited Set and Parent Map inside the Search Engine container (SLE-3, C4 Level 3) are used.

2. Decision

I decided to use Breadth-First Search as the primary solver: a FIFO queue as the Frontier, a visited set to avoid revisiting cells, and a parent map to rebuild the path at the end. DFS (LIFO stack) is kept only as a baseline for comparison, and both algorithms share the same Neighbour Generator.

3. Alternatives Considered

Alternative 1: Depth-First Search (stack). Implemented and profiled in SLE-2. Rejected as the primary solver because its paths were 2.0× to 2.7× longer and it gives no shortest-path guarantee.

Alternative 2: A* with Manhattan-distance heuristic. Considered but not implemented. It would typically expand fewer nodes than BFS while staying optimal on this unweighted grid, but it needs a priority queue and a new Heuristic component, which went beyond the BFS-vs-DFS comparison of this project. Kept as a future upgrade (SLE-3, Section 6).

Alternative 3 (optional): Greedy Best-First Search. Considered only. It is fast but does not guarantee the shortest path, and it also needs a heuristic.

4. Rationale / Justification

Path quality is the output. In SLE-2, BFS paths were 39, 79 and 141 cells against DFS's 79, 193 and 379 (2.0×, 2.4×, 2.7× longer), and the gap widens with maze size.

The extra cost is small. At 70×70, BFS took 3.084 ms and 3,167 nodes against DFS's 0.664 ms and 709 nodes, only about 2.4 ms more.

No dependence on luck. DFS's node count depends on layout and neighbour order (SLE-2, Section 4), while BFS gives a shortest path on an unweighted grid by property.

Predictable growth. From 20×20 to 70×70, cells grew about 12×, BFS nodes about 10.5× (302 → 3,167) and time about 11× (0.277 → 3.084 ms).

Fits the SLE-3 design. BFS and DFS differ only in the Frontier (queue vs stack), and the visited set + parent map rebuild the path, so BFS needed no extra container.

5. Consequences

Positive:

Always returns a shortest path on the unweighted grid (39, 79 and 141 cells in SLE-2), whatever the maze layout.

The design stays simple and easy to test: the same containers serve both algorithms, and no Heuristic component is needed.

Time and nodes grow in a predictable way with maze size, which makes the results easy to explain.

Negative / Trade-offs:

Slower than DFS in every SLE-2 case (about 2.7× to 4.6×) and expands about 2.9× to 4.5× more nodes.

BFS keeps a larger frontier/visited set, so memory could become the first limit on much larger mazes.

BFS is uninformed: it ignores the goal's position, and it stops being optimal if moves have different costs (A* or Dijkstra would then be needed).

The numbers come from three specific mazes and 5 runs each, so they should not be read as a universal result.

6. References (optional)

GitHub profile:

SLE-1 repository:

SLE-2 Profiling Report: Summary Comparison Table, detailed results table, Figs. 2–3 and Section 4 (Justification & Analysis); code in maze_search.py.

SLE-3 C4 Report: Fig. 2 (Container diagram), Fig. 3 (Component diagram of the Search Engine) and Section 6 (Design Decisions).

PART B: Short Summary of My SLE Journey

SLE-1: I built a basic rule-based AI Study Assistant (agent.py) with a small task manager, used GitHub Copilot Chat, and kept an AI Contribution Log. I learned to review AI code and fix its bugs, such as the logic-ordering problem and the missing bounds check. SLE-2: I moved to search and profiled BFS vs DFS on 20×20, 40×40 and 70×70 mazes using time.perf_counter and a node counter, with 5 runs per case. DFS was faster and expanded fewer nodes, but BFS gave the shortest path every time. SLE-3: I documented the system with the four C4 levels and saw that BFS and DFS differ only in how the Frontier is handled. SLE-4: this ADR records my decision to adopt BFS as the primary strategy, based on the SLE-2 numbers and the SLE-3 design.

PART C: AI Contribution Note

AI tools used (if any): Claude for this ADR. In earlier SLEs: GitHub Copilot Chat (SLE-1) and Claude (SLE-2 and SLE-3).

What AI helped with: arranging the ADR in the guideline format, drafting the wording of Context and Rationale from my SLE-1 to SLE-3 reports, calculating the ratios from my SLE-2 tables (for example 2.0×, 2.4×, 2.7× for path length) and suggesting viva points.

What I did myself: chose the decision (BFS as the primary strategy), checked every number against my SLE-2 tables and every container/component name against my SLE-3 diagrams and code, edited the text so it reflects my own work, and prepared my viva answers.

PART D: Viva Preparation Notes

Why BFS, and why not the others? The output is a path, so the shortest path matters and BFS guarantees it on an unweighted grid. DFS gave paths 2.0× to 2.7× longer. A* and Greedy need a heuristic and a priority queue, which this uninformed BFS-vs-DFS study did not need; A* is my planned upgrade.

Profiling numbers to quote (BFS / DFS). Time: 0.277 / 0.088, 1.037 / 0.380, 3.084 / 0.664 ms. Nodes: 302 / 80, 1,189 / 408, 3,167 / 709. Path: 39 / 79, 79 / 193, 141 / 379 cells.

C4 Container diagram in 1 minute. Maze Generator makes the maze; the Experiment Driver runs each algorithm 5 times; the Search Engine runs BFS or DFS using the shared Neighbour Generator; the Result Reporter times, tabulates and charts; the Output Module shows path, time, nodes and path length. Only the Frontier differs between BFS and DFS.

If the problem becomes larger. BFS nodes and time grew roughly with maze area (about 12× cells, 10.5× nodes, 11× time). Frontier/visited memory would be the first limit; for much larger or weighted mazes I would move to A* with a

Manhattan heuristic. These results are from my mazes only.

What AI generated and what I changed. SLE-1: Copilot generated the if-elif responses, the main loop and the task logic; I added the __main__ guard, fixed the fallback-ordering bug and added bounds validation. SLE-2/3 and this ADR: Claude helped with code and drafting; I verified the numbers and names, and must be able to explain every part.

Department of CSE (AI & ML) | 02AML204 — Introduction to Artificial Intelligence

SLE-4 | Record the decision, justify it, defend it.